

DEPARTMENT OF ARTIFICIAL INTELLIGENCE
AND MACHINE LEARNING

Machine Learning Operations
Course Code: 24AIM54
Lab Manual

Academic Year:2026-2027
BATCH: 2024 – 2028
YEAR: III
SEMESTER: V

Lab-Coordinators:
Dr. Umamaheshwaran
Dr. Sreejith S
Dr. Sanjeev P Kaulgud
Prof. Ramesh Prasad
Prof. Ramyashree P M
Prof. Sandhyarani S
Prof. Manoj Kumar



INSTITUTION

Vision

To emerge as an institute of eminence in the fields of engineering, technology and management in serving the industry and the nation by empowering students with a high degree of technical, managerial and practical competence.

Mission

To strengthen the theoretical, practical and ethical dimensions of the learning process by fostering a culture of research and innovation among faculty members and students.

To encourage long-term interaction between the academia and industry through their involvement in the design of curriculum and its hands-on implementation.

To strengthen and mould students in professional, ethical, social and environmental dimensions by encouraging participation in co-curricular and extracurricular activities.

To develop value based socially responsible professionals for the betterment of the society

Quality Policy

To emerge as an institute of eminence in the fields of engineering, technology and management in serving the industry and the nation by empowering students with a high degree of technical, managerial and practical competence.

Values

- Academic Freedom
- Innovation
- Integrity
- Professionalism
- Inclusiveness
- Social Responsibility

DEPARTMENT of AI & ML

Vision

To develop an outstanding AI and ML professionals with profound practical, research & managerial skills to meet ever changing Industrial Social and Technological needs of the Society.

Mission

- To provide strong theoretical foundations and hands-on competence in Artificial Intelligence and Machine Learning, fostering research, innovation, and technical excellence in alignment with industry and national needs.
- To establish sustainable academia–industry collaboration for curriculum enrichment, real-time problem solving, internships, and emerging AI technology implementation.
- To develop ethically responsible, socially conscious, and environmentally aware AI professionals through holistic learning and active participation in co-curricular and professional activities.

Program Educational Objectives (PEOs)

PEO1	Graduates will build successful careers in Artificial Intelligence and Machine Learning by applying strong theoretical foundations, analytical skills, and modern tools to solve complex industrial and societal problems.
PEO2	Graduates will pursue higher education, research, entrepreneurship, or leadership roles in emerging AI technologies through continuous learning, innovation, and industry collaboration.
PEO3	Graduates will demonstrate ethical responsibility, environmental awareness, and social consciousness while developing and deploying AI solutions for sustainable societal impact.

Program Specific Outcomes (PSOs)

A graduate of the Artificial Intelligence and Machine Learning Program will demonstrate

PSO1: Ability to design, develop, and deploy intelligent systems using machine learning algorithms, deep learning architectures, data analytics, and AI frameworks to address real-world applications.

PSO2: Ability to analyse large-scale data, interpret model outcomes, and implement responsible, secure, and ethical AI solutions aligned with industry standards and societal needs.

Knowledge and Attitude Profile (WK)

WK1: A systematic, theory-based understanding of the natural sciences applicable to the discipline and awareness of relevant social sciences.

WK2: Conceptually-based mathematics, numerical analysis, data analysis, statistics and formal aspects of computer and information science to support detailed analysis and modelling applicable to the discipline.

WK3: A systematic, theory-based formulation of engineering fundamentals required in the engineering discipline.

WK4: Engineering specialist knowledge that provides theoretical frameworks and bodies of knowledge for the accepted practice areas in the engineering discipline; much is at the forefront of the discipline.

WK5: Knowledge, including efficient resource use, environmental impacts, whole-life cost, reuse of resources, net zero carbon, and similar concepts, that supports engineering design and operations in a practice area.

WK6: Knowledge of engineering practice (technology) in the practice areas in the engineering discipline.

WK7: Knowledge of the role of engineering in society and identified issues in engineering practice in the discipline, such as the professional responsibility of an engineer to public safety and sustainable development.

WK8: Engagement with selected knowledge in the current research literature of the discipline, awareness of the power of critical thinking and creative approaches to evaluate emerging issues.

WK9: Ethics, inclusive behavior and conduct. Knowledge of professional ethics, responsibilities, and norms of engineering practice. Awareness of the need for diversity by reason of ethnicity, gender, age, physical ability etc. with mutual understanding and respect, and of inclusive attitudes.

Program Outcomes (POs)

UG – Engineering Program (August 2024)

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Machine Learning Operations

Course Code	**24AIM54**	**CIE Marks**	**50**
L:T:P:S	**3:0:0:0**	**SEE Marks**	**50**
Hrs / Week	**03**	**Total Marks**	**100**
Credits	**03**	**Exam Hours**	**03**

Course outcomes:

At the end of the course, the student will be able to:

24AIM54.1	Apply the concepts, principles and tools of DevOps, including version control systems, Git, GitHub and Continuous Integration practices.
24AIM54.2	Apply containerization techniques using Docker for application deployment and management.
24AIM54.3	Analyze the machine learning lifecycle and identify the role of MLOps in developing reliable and scalable ML systems.
24AIM54.4	Implement MLOps practices for data management, experiment tracking, model training and validation.
24AIM54.5	Design and develop automated machine learning pipelines using CI/CD, workflow orchestration and cloud-based MLOps platforms.
24AIM54.6	Deploy, monitor and maintain machine learning models in production environments using AWS, containers and model monitoring techniques.

Mapping of Course Outcomes to Program Outcomes and Program Specific Outcomes:

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2
24AIM54.1	3	2	-	-	2						2	3	2
24AIM54.2	3	3	2	1	3						2	3	3
24AIM54.3	2	2	3	2	3						2	2	2
24AIM54.4	2	3	3	2	3						2	2	3
24AIM54.5	2	2	3	2	3						2	2	2
24AIM54.6	3	3	3	3	3						2	3	3

MODULE-1	**DEVOPS FUNDAMENTALS AND VERSION CONTROL**	**24AIM54.1**	**8 Hours**

Introduction to DevOps, Software Development Life Cycle (SDLC), Agile and DevOps, Version Control Concepts, Installation and Configuration of Git, Git Commands (init, add, commit, push, pull, clone, merge, branch), Git hub actions for workflow automations.

Hands on:

1. You have been assigned to develop a simple portfolio webpage for a client. Before writing code, prepare your development environment following DevOps practices and maintain the project using Git version control.
2. Publish a local Git repository to GitHub and synchronize project versions between the local system and the remote repository.
3. Demonstrate collaborative development by publishing your project to GitHub and implement a feature using branching and merge multiple features into a common project.
4. Automate Continuous integration (CI) using GitHub Actions whenever code is pushed to GitHub.

MODULE-2	**CONTAINERIZATION AND ORCHESTRATION**	**24AIM54.2**	**8 Hours**

Docker Fundamentals, Docker Architecture, Docker Installation and Configuration, Docker Images and Containers, Dockerfile Creation, Docker Hub and Container Registry, Deployment of Python Flask Applications using Docker, Introduction to Kubernetes, Container Orchestration Concepts.

Hands on:

1. Develop a simple Student Management REST API using Python Flask that allows users to retrieve and add student records. Test the implemented APIs using Postman.
2. Explore the advantages of containerization by creating a Docker image and executing a Python Flask application inside a Docker container.
3. Store the application as a Docker image and distribute it using Docker Hub.
4. Pull a docker image from dockerhub repository and run it locally.

MODULE-3	**MLOPS FOUNDATIONS**	**24AIM54.3, 24AIM54.4**	**8 Hours**

Introduction to Machine Learning Operations (MLOps), DevOps vs MLOps, Machine Learning Lifecycle, Challenges in Production ML Systems, Introduction to MLflow, Experiment Tracking, Model Registry and Versioning, Experiment Comparison, Experiment Retrieval and Reproducibility.

Hands on:

1. Develop a machine learning model to predict house prices based on property features. Configure MLflow and explore its experiment tracking interface.
2. Train a machine learning model with different hyperparameters and preprocessing techniques. Use MLflow to track all experiments systematically.
3. Execute several experiments on a ML model. Identify the best model based on evaluation metrics and maintain it using the MLflow Model Registry.
4. Use MLflow to retrieve a saved experiment details and reproduce the model execution.

Case Study / Applications	Build a Continuous Training (CT) pipeline that automatically retrains a machine learning model when the training dataset is updated using Github Actions.

MODULE-4	**AUTOMATED ML PIPELINES**	**24AIM54.5**	**8 Hours**

Building REST APIs for Machine Learning using Flask, Streamlit and Docker for Machine Learning Models, Introduction to MLOps Pipelines, Automated Model Validation, Automated Docker Image Build and Deployment Workflow, Model Deployment Strategies, Continuous Integration/Continuous Deployment (CI/CD) for Machine Learning using GitHub Actions.

Hands on:

1. Develop a Flask-based REST API to expose a trained ML model for prediction requests through external applications like postman.
2. Develop a Streamlit-based user-friendly web interface for a ML model that accepts user inputs, displays prediction results, and package the complete application using Docker for portable deployment.
3. Implement direct deployment strategy by replacing an existing Docker container with an updated version of a machine learning application.
4. Configure a GitHub Actions workflow to automate the process of building and validation of the Docker image of a ML project whenever changes are pushed to the repository.

Case Study / Applications	Study of Different model deployment strategies.

MODULE-5	**MODEL DEPLOYMENT & MONITORING**	**24AIM54.5 24AIM54.6**	**8 Hours**

Introduction to Cloud-Based Machine Learning Deployment, AWS EC2 servers, Creating and Configuring Virtual Machine Instances, Configuring Docker on Cloud Servers, Deploying Containerized Machine Learning Applications on AWS, Batch and Real-Time Inference, Overview of Kubernetes for Machine Learning Workloads, Model Drift Detection, Data Drift Monitoring.

Hands on:

1. Create and configure an AWS EC2 virtual machine and host a containerized application.
2. Deploy a containerized machine learning application on an AWS EC2 instance using Docker.
3. Verify deployed application accessibility through the server's public IP address.
4. Deploy a containerized machine learning application on a Kubernetes cluster hosted on an AWS EC2 instance.

Case Study / Applications	Comparison study of Batch and Real-Time Inference workflows, Explore NGINX as reverse proxy

CIE Assessment Pattern (50 Marks – Hands On) –

RBT Levels		Marks Distribution	
		Test (s)	Online Learning / Certification
		25	25
L1	Remember	5	-
L2	Understand	5	5
L3	Apply	5	15
L4	Analyze	5	5
L5	Evaluate	5	-
L6	Create	-	-

SEE Assessment Pattern (50 Marks – Theory)

RBT Levels		Exam Marks Distribution (50)
L1	Remember	10
L2	Understand	10
L3	Apply	20
L4	Analyze	10
L5	Evaluate	--
L6	Create	--

Suggested Learning Resources:

Text Books:

1) Practical MLOps, Operationalizing Machine Learning Models Noah Gift, Alfredo Deza, O'Reilly Media 2021.
2) D. Kreuzberger, N. Kühl and S. Hirschl, "Machine Learning Operations (MLOps): Overview, Definition, and Architecture," in IEEE Access, vol. 11, pp. 31866-31879, 2023. doi: 10.1109/ACCESS.2023.3262138.
3) Ameisen, E. (2020). Building machine learning powered applications: Going from idea to product. [O'Reilly Media]. ISBN: 978-1492045113.
4) Emmanuel Ameisen, "Building Machine Learning Powered Applications: Going from Idea to Product", O'Reilly Media, Inc., 2020.

Reference Books:

1) Sanjeev Sharma, "The DevOps Adoption Playbook: A Guide to Adopting DevOps in a Multi-Speed IT Enterprise", Wiley, 2017.
2) Carl Osipov, "MLOps Engineering at Scale", O'Reilly Media, Inc., 2022.

Web links and Video Lectures (e-Resources):

- IIT Kharagpur: https://www.youtube.com/@MLOpsIITMBSDegree
- Great Learning: https://www.youtube.com/watch?v=PMRIXTmeXDc
- IBM: https://www.youtube.com/watch?v=OejCJL2EC3k

Activity-Based Learning (Suggested Activities in Class)/ Practical Based learning

- Online classes using Jeopardy Lab
- Contents related activities (Activity-based discussions)

➢ For active participation of students, instruct the students to read research papers on Big Data Analytics and have a discussion.

➢ Presentations

LAB RUBRICS

Internal Assessment Marks: 50

Divided into three components:

- Global certification: 25 marks
- Internal Test: 25 marks

Global certification: 20 marks

Each student will be completing institution approved Global certification Course. Marks out of 20 will be awarded on completion.

Rubric of 25 marks:

Will be carried out in every lab (for 20 lab programs)

Attributes	Descriptors	Scores
Global certification	Completes all modules and clears the certification exam	25
	Completes all the models but fails to take up exam	15
	Partially completes the course and doesn't take up exam	10
	Doesn't complete the course and no examination taken up	0

Internal Test: 25 marks

Break up of 25 marks (for each of the 2 internal tests) which is scaled down to 25 marks after the conduction of 2 internal tests:

The 1st lab internal will comprise of the first 10 lab programs and the 2nd lab internal will comprise of the next 10 lab programs.

Attributes	Descriptors	Scores
Wtrite up (5)	Writes all steps clearly with clear explanations; Writes output and a clear conclusion on the obtained results	5
	Writes partial steps; Writes output and conclusion on the obtained results	3-4
	Writes few steps; Writes expected results of the experiment	1-2
	Doesnt write any steps / results	0
Execution & Results (10)	Executes the experiment correctly with all required steps and demonstrates the result.	10
	Executes the experiment with minor errors and demonstrates the result	5-9
	Partially executes the experiment; obtains an incomplete or partially correct result. Explains the expected results	1-4
	Unable to execute the experiment correctly; major implementation errors and not able to explain the expected output.	0
Viva Voce(10)	Answers 100% questions correctly	10
	Answers 75% questions correctly	5-9
	Answers satisfactorily	1-4
	Does not answer any question	0

SEE

Assessment Marks: 50

Session End Examination is conducted for 50 marks as project presentation

Attributes	Descriptors	Scores
Program Write- up (10)	Writes the project description, Design details, software tools used and brief description on MLOPS concept implemented in the project	10
	Writes the project description, Design details, software tools used and MLOPS concept implemented partially	6-9
	Writes about the MLOPS concepts used with less detail and clarity	1-5
	Doesnt write any details on the project implemented	0
Demo (30)	Implemented and demonstrated all MLOPS concepts in the syllabus	30
	Implemented and demonstrated few MLOPS concepts in the syllabus	20-29
	Incomplete implementation / has not incorporated any of the MLOPS concepts	1-19
Viva Voce (10)	Answers questions correctly and confidently; demonstrates clear understanding of project implemented	10
	Answers most questions satisfactorily and demonstrates a good understanding of the project and related concepts.	6-9
	Answers only basic questions; demonstrates limited understanding of the concepts and implementation.	1-5
	Unable to answer the questions or demonstrates no meaningful understanding of the project	0

Module 1: DEVOPS FUNDAMENTALS AND VERSION CONTROL

1. You have been assigned to develop a simple portfolio webpage for a client. Before writing the application code, prepare the development environment following DevOps practices and maintain the project using Git version control.

1. Aim

To prepare a development environment for developing a simple portfolio webpage and manage the project using Git version control by following basic DevOps practices.

2. Objective

After completing this experiment, the student will be able to:

- Set up a suitable development environment for web application development.
- Create and organize a project directory following standard practices.
- Initialize a Git repository.
- Track changes made to project files.
- Create commits with meaningful messages.
- View the project history using Git.
- Understand the importance of version control in a DevOps workflow.

3. Software and Tools Required

- Operating System: Windows/Linux/macOS
- Visual Studio Code or any suitable code editor
- Git
- Web browser
- Command Prompt / PowerShell / Terminal

4. Procedure

Step 1: Verify Git Installation

Open Command Prompt, PowerShell, or Terminal and execute:

git --version

If Git is not installed, install Git before proceeding with the experiment.

Step 2: Configure Git

Configure the username and email address that will be associated with the commits.

git config --global user.name "Student Name"

git config --global user.email "student@example.com"

Verify the configuration using:

git config --global --list

The configured username and email address should be displayed.

Step 3: Create the Project Directory and open the code editor

Create a directory for the portfolio project.

For example:

mkdir portfolio-website

Move into the project directory:

cd portfolio-website (The project directory will serve as the working directory for the application.)
If Visual Studio Code is installed, execute: **code .**

Step 4: Create the Initial Project Structure

Create the following basic project structure:

```
portfolio-website/
│
├── index.html
└── README.md
```

Step 5: Initialize a Git Repository

Inside the project directory, execute:

git init

A Git repository will be created inside the project directory. This converts the project directory into a Git-managed project.

Step 6: Check the Repository Status

Execute:

git status

Git will display the files that are currently untracked.

Example:

Untracked files:
  index.html
  README.md

Step 7: Create a .gitignore File

Create a file named:

.gitignore

The .gitignore file specifies files or folders that should not be tracked by Git.

This is an important practice because unnecessary editor-specific, operating-system-specific, or temporary files should not be included in the repository.

Step 8: Stage the Project Files

Add the required project files to the Git staging area:

git add .

Verify the status:

git status

The files should now appear under Changes to be committed.

Step 9: Create the Initial Commit

Create the first version of the project using a meaningful commit message:

git commit -m "Initial project setup"

A commit represents a saved version of the project at a particular point in time.

The commit message should clearly describe the changes introduced.

Step 10: Make a Change to the Project

Open index.html and add a basic heading, for example:

```
<!DOCTYPE html>
<html>
<head>
  <title>My Portfolio</title>
</head>
<body>
  <h1>My Portfolio</h1>
</body>
</html>
```

Save the file.
Now check the repository status:
git status
Git should identify index.html as a modified file.

Step 11: Review the Changes

Execute:
git diff
The command displays the changes made to files since the previous commit.
This allows the developer to review modifications before committing them.

Step 12: Stage and Commit the Changes

Stage the modified file:
git add index.html
Commit the change:
git commit -m "Add initial portfolio page"
The repository now contains two versions:
Commit 1 → Initial project setup
Commit 2 → Add initial portfolio page

Step 12: View Commit History

Execute:
git log --oneline
Example output:
a82f4c1 Add initial portfolio page
71b23d8 Initial project setup
This provides a concise history of the project.

Step 12: Create a Git Branch

Create a separate branch for further development. The use of branches allows developers to work on new features without directly affecting the stable version of the project:
git branch feature-homepage
Switch to the branch:
git switch feature-homepage
or simply, **git switch -c feature-homepage**

Step 13: Verify the Current Branch

Execute:

git branch

The currently active branch will be indicated by *.

Step 14: Modify the Portfolio Page and commit to the new branch.

Open index.html and add a new section, for example:

```
<section>
  <h2>Welcome to My Portfolio</h2>
  <p>This section is being developed as part of a new homepage design.</p>
</section>
```

Save the file.

Open the webpage in the browser and observe the change.

Check the repository status:

git status

Stage the modified file:

git add index.html

Commit the change:

git commit -m "Add new homepage design"

The new commit now belongs to the feature-new-design branch.

Step 15: Switch Back to the Stable Branch

Switch back to the main branch (or master branch):

git switch main

Open index.html again in the browser.

Observe that the stable version remains unchanged even though the new design was committed in the feature-new-design branch.

Viva-Voce Questions

1. What is DevOps?
2. What is the purpose of Git?
3. What is the difference between Git and GitHub?
4. What is a Git repository?
5. What is the purpose of git init?
6. What is the difference between git add and git commit?
7. What is the purpose of git status?
8. What information is provided by git log?
9. Why is .gitignore used?
10. What is a Git branch?
11. Why should development work be performed using branches?
12. What is the difference between a working directory, staging area, and repository?
13. Why should meaningful commit messages be used?
14. How does version control support DevOps practices?
15. What would happen if a developer made changes without using version control?

2. Publish a local Git repository to GitHub and synchronize project versions between the local system and the remote repository.

1. Aim

To publish a local Git repository to GitHub and synchronize project versions between the local development environment and the remote repository. To enable remote storage, collaboration, backup, and access to different project versions, the local Git repository must be published to GitHub.

2. Objectives

After completing this experiment, the student will be able to:

- Create a remote repository on GitHub.
- Connect a local Git repository to a remote repository.
- Push local commits to GitHub.
- Retrieve changes from a remote repository.
- Synchronize local and remote project versions.
- Understand the relationship between a local repository and a remote repository.

3. Procedure

Step 1: Create a Repository on GitHub

Open GitHub in a web browser and sign in to your account or create one if not available.

Create a new repository using the following steps:

1. Select New repository.
2. Enter a suitable repository name, for example: portfolio-website
3. Provide a short description if required.
4. Select the appropriate repository visibility.
5. Do not initialize the repository with a README, .gitignore, or license, because these files already exist in the local repository.
6. Select Create repository. A new empty remote repository will be created.

Step 2: Connect the Local Repository to GitHub

GitHub provides a repository URL after creating the repository.

Add this remote repository to the local project using:

git remote add origin <GitHub-Repository-URL>

For example:

git remote add origin https://github.com/username/portfolio-website.git

Here:

- origin is the conventional name given to the remote repository.
- The URL identifies the GitHub repository.

Step 3: Verify the Remote Repository

Execute:

git remote -v

The output should display the GitHub repository URL.

Example:

origin https://github.com/username/portfolio-website.git (fetch)

origin https://github.com/username/portfolio-website.git (push)

This confirms that the local repository is connected to GitHub.

Step 4: Check the Current Branch

In the terminal of the code editor, execute:

git branch

If the current branch is main, proceed to the next step.

If the branch is named master, it can be renamed to main using:

git branch -M main

Check the repository status and logs: The repository should be clean or should contain only the changes that are intentionally being developed. All files that are to be put in the GitHub repository must be committed.

Step 5: Push the Local Repository to GitHub

Push the local main branch to the GitHub repository:

git push -u origin main

Step 6: Verify the Repository on GitHub

Open the GitHub repository in a browser.

Verify that the following are visible:

portfolio-website/

```
│
├── index.html
├── README.md
```

Step 7: Synchronizing a Local Change with GitHub

Step 1: Make a Change Locally in index.html

Step 2: Stage and Commit the Local Change (At this point, the change exists in the local repository, but it has not yet been uploaded to GitHub)

Step 3: **Push the Change to GitHub**

Execute: **git push**

The new commit will be transferred to GitHub.

Refresh the GitHub repository page and verify that the updated index.html and the new commit are available.

Step 8: Synchronizing a Remote Change with the Local Repository

Step 1: Make a Change on GitHub

Open the README.md file in the GitHub repository.

Use the GitHub web interface and add some text content (The change now exists in the remote repository, but the local repository does not yet contain it.)

Step 2: Retrieve the Remote Changes

Return to the terminal and execute:

git pull

Git will download the changes from GitHub and integrate them into the local repository.

Step 9: Verify the Change Locally

Open README.md in Visual Studio Code.

The content added through GitHub should now be available in the local project.

Viva-Voce Questions

1. What is GitHub?
2. What is the difference between Git and GitHub?
3. What is a remote repository?
4. What is the purpose of git remote add origin?
5. What does origin represent?
6. What is the purpose of git push?
7. What is the purpose of git pull?
8. What is the difference between git fetch and git pull?
9. What happens when a local commit is created but git push is not executed?
10. Can changes made on GitHub automatically appear in the local repository?
11. What is an upstream branch?
12. What is the purpose of git clone?
13. Why is GitHub useful in a DevOps workflow?
14. How does a remote repository support collaboration among developers?
15. What happens if two developers modify the same part of a file and try to synchronize their changes?

3. Demonstrate collaborative development by publishing your project to GitHub and implement a feature using branching and merge multiple features into a common project.

1. Aim

To demonstrate collaborative development by publishing a project to GitHub, implementing multiple features using separate branches, and merging the features into a common project.

2. Objectives

After completing this experiment, the student will be able to:

- Create and manage branches in a Git repository.
- Develop different features independently using branches.
- Commit changes to individual branches.
- Merge multiple branches into the main branch.
- Understand how branching supports parallel development.
- Verify the final integrated version of a project.
-

3. Procedure

Step 1: Open an existing project or create a new one and check repository status to make sure the tree is clean before any activity.

Step 2: Synchronize with GitHub

git switch main
git pull

Part A – Develop Feature 1

Step 3: Create the First Feature Branch, switch to it and implement a new feature.

Create a branch for the first feature:
git switch -c feature-about
Open index.html and add an About Me section.
For example:

```
<section>
  <h2>About Me</h2>
  <p>
    I am an AIML student interested in artificial intelligence,
    machine learning and software development.
  </p>
</section>
```

Save the file.
Open the webpage in a browser and verify that the new section is displayed correctly.

Step 4: Stage and commit Feature 1

git add index.html
git commit -m "Add About Me section"

Step 5: Push this new branch to GitHub

git push -u origin feature-about
Open the GitHub repository and verify that the feature-about branch is available.

Part B – Develop Feature 2

Step 6: Repeat step 6 by creating a new branch and implementing another feature.

Switch back to main:

git switch main

Create another branch:

git switch -c feature-skills

Implement Feature 2

Open index.html and add a Skills section:

```
<section>
  <h2>Skills</h2>
  <ul>
    <li>Python</li>
    <li>Machine Learning</li>
    <li>Web Development</li>
  </ul>
</section>
```

Save and test the webpage.

Step 7: Stage and commit it.

git add index.html

git commit -m "Add Skills section"

Step 8: Push this branch to GitHub

git push -u origin feature-skills

Part C – Merge Feature 1

Step 9: Switch to the Main Branch

Switch to the stable branch:

git switch main

Step 10: Merge Feature 1

git merge feature-about

Verify the webpage. The **About Me** section should now be visible.

Step 11: Push the Updated Main Branch

Upload the updated main branch to GitHub:

git push

Part D – Merge Feature 2

Step 12: Repeat the same steps for the other feature.

Step 13: View the Commit History

Execute:

git log --oneline --graph --all

The command displays the development history and demonstrates that the features were developed in separate branches and subsequently merged into main.

Viva-Voce Questions

1. What is the purpose of creating a branch?
2. What is the difference between main and a feature branch?
3. Does a commit on a feature branch immediately modify main?
4. What is the purpose of git switch?
5. What is the purpose of git merge?
6. Why should features be developed in separate branches?
7. What happens when a feature branch is merged into main?
8. What is the purpose of git push after merging?
9. What is a merge conflict?
10. How can the development history of different branches be viewed?

4. Automate Continuous integration (CI) using GitHub Actions whenever code is pushed to GitHub.

1. Aim

To automate Continuous Integration (CI) for a static website using GitHub Actions and observe the automatic execution of a workflow whenever changes are pushed to GitHub.

2. Objectives

After completing this experiment, the student will be able to:

- Create and test a static website locally.
- Host a static website using GitHub Pages.
- Create a basic GitHub Actions workflow.

3. Procedure

Part A – Create and Test the Website

Step 1: Create the Project

Open Visual Studio Code and create a folder a new folder
Create an index.html file containing a simple portfolio webpage.

Step 2: Run the Website Locally

Open the Extensions panel in Visual Studio Code. Search for: Live Server. Install the extension.
Open index.html → Right-click → **Open with Live Server**.
Verify that the portfolio webpage opens successfully in the browser.

Part B – Publish and Host the Website

Step 3: Publish the Project

Initialize the local Git repository, create the initial commit, create a GitHub repository, connect the local repository to GitHub, and push the project to the main branch.

Step 4: Enable GitHub Pages

Open **GitHub Repository → Settings → Pages**, under build and deployment, select the main branch and / (root), and save the configuration.
Wait for deployment to complete. Once completed, open the generated GitHub Pages URL.
Observe that the portfolio website is accessible online through github hosting.

Part C – GitHub Actions for Continuous Integration

Step 5: Modify the Website

Return to Visual Studio Code.
Modify index.html. Save changes.

Step 6: Test Locally

Run the webpage using Live Server. Verify that the new content appears correctly on the local webpage.

Step 7: Commit and Push

Use the Git procedure learned in the previous experiments:
commit the changes with an appropriate message, and push them to the main branch.

Part G – Observe Automatic CI Execution

Step 8: Open Actions

Go to:

GitHub → Actions

A new workflow run should appear automatically. Once the process is complete, Open the GitHub Pages URL created earlier. Refresh the page.

Observe that the latest webpage version will be available online.

Viva-Voce qustions:

1. What is Continuous Integration?
2. What is GitHub Actions?
3. What is the purpose of ci.yml?
4. What does on: push indicate?
5. Why does git push trigger the CI workflow?
6. Does git commit trigger the workflow? Why?
7. What is the purpose of actions/checkout?
8. What is the purpose of runs-on?
9. What is GitHub Pages?
10. What is the difference between GitHub Pages and GitHub Actions?
11. What happens when a CI check fails?
12. Why is automated validation useful in software development?
13. What is the role of the main branch in this experiment?
14. What is the difference between Live Server and GitHub Pages?

MODULE-2: CONTAINERIZATION AND ORCHESTRATION

1. Develop a simple Student Management REST API using Python Flask that allows users to retrieve and add student records. Test the implemented APIs using Postman.

1. Aim

To develop a simple Student Management REST API using Python Flask that allows users to retrieve and add student records, and to test the implemented REST APIs using Postman.

2. Objectives

After completing this experiment, the student will be able to:

- Set up a Python Flask development environment.
- Develop a basic REST API using Flask.
- Implement GET and POST API endpoints.
- Exchange data using JSON.
- Run a Flask-based API locally.
- Test REST APIs using Postman.

3. Software and Tools Required

- Visual Studio Code
- Python
- Python Flask
- Postman
- Web browser
- Command Prompt / VS Code Terminal

4. Procedure

Step 1: Open Visual Studio Code

Launch **Visual Studio Code**.

Create a new folder named student-api and launch VS Code here.

Step 2: Open the Terminal

Open: **Terminal → New Terminal**

Step 3: Verify Python Installation

Execute:

python --version

Step 4: Create a Virtual Environment

Create a Python virtual environment:

python -m venv venv

Step 5: Activate the Virtual Environment

Windows

venv\Scripts\activate

Linux/macOS

source venv/bin/activate

Step 6: Install Flask

Install Flask using:

pip install flask

Step 7: Create the Python File

Create a file named:

app.py

The project structure should now be:

```
student-api/
│
 ├── app.py
└── venv/
```

Add the following code to app.py or generate using prompt.

```
from flask import Flask, request, jsonify
app = Flask(__name__)

students = [
   {
      "usn": "A01",
      "name": "Rahul",
      "department": "AIML",
      "semester": 5
   },
   {
      "usn": "A02",
      "name": "Priya",
      "department": "AIML",
      "semester": 5
   }
]


@app.route('/students', methods=['GET'])
def get_students():
   return jsonify(students)


@app.route('/students', methods=['POST'])
def add_student():
   student = request.get_json()
   students.append(student)

   return jsonify({
      "message": "Student added successfully",
      "student": student
   }), 201
```

```
if __name__ == '__main__':
    app.run(debug=True)
```

Step 8: Start the Flask Application

In the terminal, execute:

python app.py

Step 9: Open Postman

Launch Postman.

Create a new HTTP request.

Select:

GET

Enter:

http://127.0.0.1:5000/students

Click Send

Step 10: Create a POST Request

In Postman, create another request.

Select:

POST

Enter:

http://127.0.0.1:5000/students

Select:

Body → raw → JSON

Enter:

```
{
   "usn": "A03",
   "name": "Anita",
   "department": "AIML",
   "semester": 5
}
```

Click Send. This posts the data to the application.

Viva Questions:

1. What is Flask?
2. What is a REST API?
3. What is the purpose of the GET method?
4. What is the purpose of the POST method?
5. What is JSON?
6. What is Postman?
7. What is an API endpoint?
8. What is jsonify() in Flask?
9. What does request.get_json() do?

2. Explore the advantages of containerization by creating a Docker image and executing a Python Flask application inside a Docker container.

1. Aim

To explore the advantages of containerization by creating a Docker image and running a Python Flask application inside a Docker container.

2. Objectives

- To understand the concept of containerization.
- To create a Dockerfile for a Flask application.
- To build a Docker image.
- To run a Flask application inside a Docker container.
- To access the Flask application through a web browser.
- To understand the advantages of Docker containers.

3. Software and Tools Required

Python
Flask
Docker Desktop / Docker Engine
Visual Studio Code
Web Browser
Command Prompt / PowerShell / Terminal

4. Procedure

Step 1: Create a Project Folder

Create a folder named:

```
flask-docker-app
```

Open the folder in Visual Studio Code.

You can also use:

```
mkdir flask-docker-app
cd flask-docker-app
code .
```

Step 2: Create app.py

Create a file named:

```
app.py
```

Add the following code:

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return '<h1>Hello from Flask Docker Container!</h1>'

@app.route('/about')
def about():
    return '<h2>This Flask application is running inside Docker.</h2>'

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Step 3: Create requirements.txt

Create a file named:

requirements.txt

Add:

Flask

Step 4: Create a Dockerfile

Create a file named: Dockerfile

Add the following code:

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

Step 5: Create .dockerignore

Create a file named:

.dockerignore

Add:

```
venv/
__pycache__/
*.pyc
.git/
.gitignore
```

This prevents unnecessary files from being copied into the Docker image.

Step 6: Check Docker Installation

Open the terminal and execute:

```
docker –version
```

If Docker is installed correctly, its version will be displayed.

Also check whether Docker is running:

```
docker info
```

Step 7: Build the Docker Image

Make sure you are inside the project folder.

Run:

```
docker build -t flask-docker-app .
```

Here:

- docker build → Creates a Docker image.
- -t flask-docker-app → Gives the image a name.
- . → Uses the current folder as the build context.

After successful execution, the Docker image will be created.

Step 8: Check the Docker Image

Run:

```
docker images
```

You should see an image similar to:

```
REPOSITORY          TAG       IMAGE ID       CREATED          SIZE
-docker-app         latest    xxxxxxxxxxxx   A few seconds ago
```

Step 9: Run the Docker Container

Run:

```
docker run -d -p 5000:5000 --name flask-container flask-docker-app
```

Here:

- -d → Runs the container in the background.
- -p 5000:5000 → Maps the host port to the container port.
- --name flask-container → Gives the container a name.
- flask-docker-app → Specifies the Docker image.

Step 10: Check the Running Container

Run:

```
docker ps
```

You should see:

```
CONTAINER ID   IMAGE            PORTS
xxxxxxxxxxxx   flask-docker-app   0.0.0.0:5000->5000/tcp
```

This confirms that the Flask application is running inside the Docker container.

Step 11: Open the Application in a Browser

Open a web browser and enter:

```
http://localhost:5000
```

Expected output:

```
Hello from Flask Docker Container!
```

Step 12: Test the About Page

Open:

```
http://localhost:5000/about
```

Expected output:

```
This Flask application is running inside Docker.
```

Step 13: View Container Logs

To view the Flask application's output, run:

```
docker logs flask-container
```

The logs will show information about the Flask server and incoming requests.

Step 14: Stop the Container

After testing the application, stop the container:

```
docker stop flask-container
```

Step 15: Remove the Container

To remove the stopped container:

```
docker rm flask-container
```

7. Important Docker Commands

Command	Purpose
docker --version	Check Docker installation
docker build -t flask-docker-app .	Build Docker image
docker images	List Docker images
docker run -d -p 5000:5000 --name flask-container flask-docker-app	Run container
docker ps	Show running containers
docker logs flask-container	View container logs
docker stop flask-container	Stop container
docker rm flask-container	Remove container

9. Result

The Python Flask application was successfully containerized using Docker. A Docker image was created using a Dockerfile, and the image was executed as a Docker container. The Flask application was successfully accessed through a web browser using localhost:5000.

Thus, the advantages and basic implementation of **containerization using Docker** were successfully demonstrated.

10. Viva-Voce Questions

1. What is Docker?
2. What is containerization?
3. What is a Docker image?
4. What is a Docker container?
5. What is a Dockerfile?
6. What is the purpose of the FROM instruction?
7. What is the purpose of the WORKDIR instruction?
8. What is the purpose of the COPY instruction?
9. What is the purpose of the EXPOSE instruction?
10. What is the purpose of the CMD instruction?
11. What is the difference between a Docker image and a Docker container?
12. What is the purpose of docker build?
13. What is the purpose of docker run?
14. What is port mapping in Docker?
15. What are the advantages of containerization?

3.Store the application as a Docker image and distribute it using Docker Hub.

1.Aim

To containerize the Student Management REST API using Docker, create a Docker image for the application, and distribute the image using Docker Hub.

2. Objectives

- To create a Dockerfile for the Flask application.
- To build a Docker image for the application.
- To run the application inside a Docker container.
- To test the Dockerized Flask application.
- To create a Docker Hub repository.
- To push and distribute the Docker image using Docker Hub.

3. Software and Tools Required

Python
Flask
Docker Desktop
Docker Hub
Visual Studio Code
Command Prompt / PowerShell / Terminal
Web Browser

4. Procedure

Step 1: Check Docker Installation

Open Command Prompt or Terminal and enter:

```
docker --version
```

If Docker is installed correctly, its version will be displayed.

Example:

```
Docker version 28.x.x
```

Step 2: Create the Project Folder

Create a folder named:

```
student-management-api
```

Open the folder in Visual Studio Code.

You can also use:

```
mkdir student-management-api
cd student-management-api
code .
```

Step 3: Create the Flask Application

Create a file named:

```
app.py
```

Add the following code:

```
from flask import Flask, jsonify, request

app = Flask(__name__)

students = [
  {
    "id": 1,
    "name": "Rahul",
    "age": 20,
```

```
        "course": "Computer Science"
    },
    {
        "id": 2,
        "name": "Priya",
        "age": 21,
        "course": "Information Technology"
    }
]


# GET API - Get all students
@app.route('/students', methods=['GET'])
def get_students():
    return jsonify(students)


# POST API - Add a new student
@app.route('/students', methods=['POST'])
def add_student():

    data = request.get_json()

    new_student = {
        "id": len(students) + 1,
        "name": data["name"],
        "age": data["age"],
        "course": data["course"]
    }

    students.append(new_student)

    return jsonify(new_student), 201


if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

The host='0.0.0.0' allows the Flask application to be accessed from outside the Docker container.

Step 4: Create requirements.txt

Create a file named:

```
requirements.txt
```

Add:

```
Flask
```

This file specifies the Python package required by the application.

Step 5: Create a Dockerfile

Create a file named:

Dockerfile

Add the following code:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

Step 6: Build the Docker Image

Open the terminal inside the project folder and run:

```
docker build -t student-management-api .
```

The Docker command reads the Dockerfile and creates an image named:

```
student-management-api
```

To check the created image, run:

```
docker images
```

Expected output will contain:

```
student-management-api
```

Step 7: Run the Docker Container

Run the Docker image using:

```
docker run -d -p 5000:5000 --name student-api student-management-api
```

Here:

-d runs the container in detached mode.

-p 5000:5000 maps port 5000 of the host to port 5000 of the container.

--name student-api gives the container a name.

Check the running container using:

```
docker ps
```

Step 8: Test the Dockerized Application

Open a web browser and enter:

```
http://localhost:5000/students
```

Expected response:

```
[
  {
    "age": 20,
    "course": "Computer Science",
    "id": 1,
    "name": "Rahul"
  },
  {
    "age": 21,
    "course": "Information Technology",
    "id": 2,
    "name": "Priya"
  }
]
```

The Flask application is now running successfully inside a Docker container.

Step 9: Create a Docker Hub Account

Open Docker Hub in a web browser and create an account if you do not already have one.

After creating the account, note your Docker Hub username.

For example:

```
Username: yourusername
```

Step 10: Login to Docker Hub

Open the terminal and run:

```
docker login
```

Enter your Docker Hub username and password when prompted.

If login is successful, Docker will display a successful login message.

Step 11: Tag the Docker Image

Before uploading the image to Docker Hub, tag it using your Docker Hub username.

Run:

```
docker tag student-management-api yourusername/student-management-api:latest
```

Replace yourusername with your actual Docker Hub username.

For example:

```
docker tag student-management-api rahul123/student-management-api:latest
```

Step 12: Verify the Tagged Image

Run:

```
docker images
```

The output should contain an image similar to:

```
REPOSITORY                          TAG
yourusername/student-management-api     latest
```

student-management-api latest

Step 13: Push the Image to Docker Hub

Run:

```
docker push yourusername/student-management-api:latest
```

Docker will upload the image to Docker Hub.

After the upload is completed, the image will be available in your Docker Hub repository.

Step 14: Pull the Image from Docker Hub

To verify that the image can be distributed, remove the local image if required and pull it again using:

```
docker pull yourusername/student-management-api:latest
```

The Docker image will be downloaded from Docker Hub.

Step 15: Run the Image from Docker Hub

Run the downloaded image using:

```
docker run -d -p 5000:5000 --name student-api-dockerhub yourusername/student-
management-api:latest
```

Check the running container:

```
docker ps
```

Then open:

```
http://localhost:5000/students
```

The Student Management API should be displayed.

7. Dockerfile

The complete Dockerfile is:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

8. Docker Commands Used

Command	Purpose
docker --version	Check Docker installation
docker build -t student-management-api .	Build Docker image
docker images	List Docker images
docker run -d -p 5000:5000 student-management-api	Run Docker container
docker ps	Display running containers
docker login	Login to Docker Hub
docker tag ...	Tag image for Docker Hub
docker push ...	Upload image to Docker Hub

docker pull ...	Download image from Docker Hub

9. Project Structure

```
student-management-api/
├── app.py
├── requirements.txt
└── Dockerfile
```

10. Docker Image Distribution

The Docker image is stored in Docker Hub using the following format:

yourusername/student-management-api:latest

Other users can download the application image using:

docker pull yourusername/student-management-api:latest

They can then run the application using:

docker run -d -p 5000:5000 yourusername/student-management-api:latest

11. Result

The Student Management REST API was successfully containerized using Docker. A Docker image was created from the Flask application, tested using a Docker container, and uploaded to Docker Hub. The Docker image was successfully distributed and could be downloaded and executed using the docker pull and docker run commands.

12. Viva-Voce Questions

1. What is Docker?
2. What is a Docker image?
3. What is a Docker container?
4. What is a Dockerfile?
5. What is Docker Hub?
6. What is the purpose of the FROM instruction in a Dockerfile?
7. What is the purpose of WORKDIR?
8. What is the purpose of COPY?
9. What is the purpose of EXPOSE 5000?
10. What is the purpose of the CMD instruction?
11. What is the difference between a Docker image and a Docker container?
12. What is the purpose of the docker build command?
13. What is the purpose of the docker run command?
14. What is the purpose of the docker push command?
15. What is the purpose of the docker pull command?
16. Why is 0.0.0.0 used in the Flask application?
17. What is port mapping in Docker?
18. Why is the Docker image tagged with a Docker Hub username?
19. How can another user run the application from Docker Hub?
20. What are the advantages of Docker?

4. Pull a docker image from dockerhub repository and run it locally.

1. Aim

To pull a Docker image from a Docker Hub repository and run the image locally using Docker.

2. Objectives

- To understand Docker Hub repositories.
- To pull a Docker image from Docker Hub.
- To verify the downloaded Docker image.
- To create and run a Docker container locally.
- To access the application running inside the container.
- To verify that the Docker image works successfully.

3. Software and Tools Required

Docker Desktop
Docker Hub
Visual Studio Code
Web Browser
Command Prompt / PowerShell / Terminal

4. Procedure

Step 1: Check Docker Installation

Open Command Prompt or Terminal and enter:

```
docker --version
```

If Docker is installed correctly, its version will be displayed.
Example:

```
Docker version 28.x.x
```

Step 2: Start Docker Desktop

Open Docker Desktop and make sure Docker Engine is running.
Docker must be running before executing Docker commands.

Step 3: Login to Docker Hub

Open the terminal and run:

```
docker login
```

Enter your Docker Hub username and password when prompted.
If the login is successful, Docker will display a successful login message.

Step 4: Select a Docker Hub Repository

Use a Docker Hub image that has already been uploaded.
For example:

```
yourusername/student-management-api:latest
```

Here:
· yourusername is the Docker Hub username.
· student-management-api is the repository name.
· latest is the image tag.
Replace yourusername with the actual Docker Hub username.

Step 5: Pull the Docker Image

Run the following command:

```
docker pull yourusername/student-management-api:latest
```

Docker will download the image from the Docker Hub repository to the local computer.
Example:

```
docker pull rahul123/student-management-api:latest
```

After the download is completed, Docker will display the downloaded image layers.

Step 6: Verify the Docker Image

To view the downloaded Docker images, run:

```
docker images
```

Expected output:

```
REPOSITORY                    TAG       IMAGE ID       CREATED
yourusername/student-management-api latest    abc123456789   ...
```

This confirms that the Docker image has been successfully downloaded to the local system.

Step 7: Run the Docker Image

Run the downloaded image using:

```
docker run -d -p 5000:5000 --name student-api yourusername/student-management-api:latest
```

Example:

```
docker run -d -p 5000:5000 --name student-api rahul123/student-management-api:latest
```

Here:

-d runs the container in detached mode.

-p 5000:5000 maps the local port 5000 to the container port 5000.

--name student-api assigns a name to the container.

Step 8: Check the Running Container

Run:

```
docker ps
```

Step 9: Test the API application using POSTMAN

10. Result

The Docker image was successfully pulled from the Docker Hub repository and stored locally. The downloaded image was successfully used to create and run a Docker container. The application running inside the container was accessed through http://localhost:5000 and tested successfully.

Viva-Voce Questions

1. What is Docker Hub?
2. What is the purpose of the docker pull command?
3. What is the difference between docker pull and docker run?
4. Where is the Docker image stored after using docker pull?
5. What is a Docker container?
6. What is the purpose of the -d option in docker run?
7. What is the purpose of port mapping in Docker?
8. What does -p 5000:5000 mean?
9. What is the purpose of the docker ps command?
10. How can you stop a Docker container?
11. How can you start a stopped Docker container?
12. How can you view the logs of a Docker container?
13. What is a Docker image tag?
14. What does the latest tag mean?
15. Can a Docker image be run on another computer?
16. What happens when docker pull is executed?
17. Why is Docker Hub useful for distributing applications?
18. How can you verify that a Docker image has been downloaded?
19. How can you access a web application running inside a Docker container?
20. What is the difference between a Docker image and a Docker container?

MODULE-3: MLOPS FOUNDATIONS

1. **Develop a machine learning model to predict house prices based on property features. Configure MLflow and explore its experiment tracking interface.**

1. Aim

To develop a machine learning model for predicting house prices using a publicly available housing dataset and track the machine learning experiments using MLflow.

2. Objectives

After completing this experiment, students will be able to:

- Configure MLflow experiment tracking.
- Record model parameters and evaluation metrics.
- Explore and compare different MLflow runs.

3. Software and Tools Required

- Visual Studio Code
- Python
- Python virtual environment
- Pandas
- Scikit-learn
- MLflow
- Web browser

4. Procedure

Step 1: Create the Project Folder

Step 2: Create and Activate the Virtual Environment

Step 3: Install Required Libraries

Install:

pandas

scikit-learn

mlflow

Step 4: Generate the Machine Learning Code to train a Linear Regression Model using a publicly available dataset like California Housing dataset / Boston Housing Dataset from scikit-learn

Simple Reference Code

The generated code should be similar to the following:

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

# Load dataset
data = fetch_california_housing()

X = data.data
```

```
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict
predictions = model.predict(X_test)

# Evaluate
mae = mean_absolute_error(y_test, predictions)
r2 = r2_score(y_test, predictions)

print("Mean Absolute Error:", mae)
print("R2 Score:", r2)
```

Step 5: Run the Program

Execute:

```
python house_price.py
```

Step 6: Generate another python program to use MLflow to track experiments.

Simple Reference Code with MLflow

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

import mlflow
import mlflow.sklearn

# Load dataset
data = fetch_california_housing()

X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# MLflow experiment
```

```
mlflow.set_experiment("House Price Prediction")

with mlflow.start_run():

    # Train model
    model = LinearRegression()
    model.fit(X_train, y_train)

    # Predict
    predictions = model.predict(X_test)

    # Evaluate
    mae = mean_absolute_error(y_test, predictions)
    r2 = r2_score(y_test, predictions)

    # Log parameters
    mlflow.log_param("test_size", 0.2)
    mlflow.log_param("random_state", 42)

    # Log metrics
    mlflow.log_metric("MAE", mae)
    mlflow.log_metric("R2", r2)

    # Log model
    mlflow.sklearn.log_model(model, "model")

    print("Mean Absolute Error:", mae)
    print("R2 Score:", r2)
```

Run and Track the Experiment

Step 7: Execute the Program

Run:

```
python house_price.py
python house_proce_mlflow.py
```

Step 8: Start the MLflow Interface

Open another terminal and execute:

```
mlflow ui
```

Open the URL displayed by MLflow in a browser.

Typically:

http://127.0.0.1:5000

Observe

The MLflow interface should display:

House Price Prediction

with at least one experiment run.

Step 9: Open the Experiment to explore the MLflow Run

Click:

House Price Prediction

Open the run and identify:

Parameters

test_size

random_state

Metrics

MAE

R2

Artifacts

The trained model should be available as an artifact.

12. Viva-Voce Questions

1. What is MLflow?
2. Why is experiment tracking important in MLOps?
3. What is an MLflow experiment?
4. What is an MLflow run?
5. What is a parameter?
6. What is a metric?
7. What is an artifact?
8. What is the purpose of mlflow.start_run()?
9. What is the purpose of mlflow.log_param()?
10. What is the purpose of mlflow.log_metric()?
11. Why are multiple runs created?
12. How can MLflow help identify the better-performing model?
13. What is the purpose of MAE?
14. What does R^2 indicate?
15. What happens if the same program is executed multiple times?
16. Why is experiment tracking important when developing machine learning models?

2. Train a machine learning model with different hyperparameters and preprocessing techniques. Use MLflow to track all experiments systematically.

1. Aim

To train a machine learning model using different hyperparameter values and preprocessing techniques and systematically track the experiments using MLflow.

2. Objectives

After completing this experiment, students will be able to:

- Understand the effect of hyperparameters on model performance.
- Apply different preprocessing techniques to input data.
- Train multiple versions of a machine learning model.
- Track parameters and metrics using MLflow.
- Compare different experimental runs.

3. Procedure

Step 1: Open the Existing Project

Generate code to train a simple Linear Regression Model.

Step 2: Identify the Model and Parameters

Open the existing Python program and identify:

- Machine learning algorithm
- Input features
- Target variable
- Train-test split
- Existing model parameters
- Evaluation metrics

Step 3: Generate another python program to use MLflow to track experiments.

Step 4: Modify the Linear Regression Program.

Modify the existing house price prediction program to conduct experiments with different preprocessing techniques and model hyperparameters.

For example, Use StandardScaler and MinMaxScaler as two preprocessing alternatives. Use the same regression model for both experiments, but allow one suitable hyperparameter of the model to be changed.

Experiment 1 → StandardScaler + Hyperparameter A
Experiment 2 → MinMaxScaler + Hyperparameter A
Experiment 3 → StandardScaler + Hyperparameter B
Experiment 4 → MinMaxScaler + Hyperparameter B

Step 5: Run multiple Experiments

Run the program using different configuration combinations.

Record:

- Preprocessing technique
- Hyperparameter value
- MAE
- R^2

Step 9: Compare the Runs

Compare the different runs using the MLflow interface.

Record the results:

Run	Preprocessing	Hyperparameter	MAE	R^2
1				
2				
3				
4				

6. Expected Result

Multiple versions of the house price prediction model are successfully trained using different preprocessing techniques and hyperparameter configurations. All experiments are recorded in MLflow and their performance is compared using evaluation metrics.

8. Viva-Voce Questions

1. Why should experiments be tracked using MLflow?
2. How can MLflow be used to compare different configurations?

3. Execute several experiments on a ML model. Identify the best model based on evaluation metrics and maintain it using the MLflow Model Registry.

1. Aim

To execute multiple machine learning experiments, identify the best-performing model based on evaluation metrics, and maintain the selected model using the MLflow Model Registry.

2. Objectives

After completing this experiment, students will be able to:

- Generate multiple MLflow runs.
- Compare model performance using evaluation metrics.
- Identify the best-performing model.
- Register a trained model in the MLflow Model Registry.
- Understand different stages of model management.

3. Procedure

Step 1: Open an existing project or build a new project for a simple Linear Regression Model with MLFlow for project tracking.

Step2: Run experiments with different configurations or check MLFlow interface for previous runs.

Step 3: Determine the Selection Criterion

For this experiment, use **R^2 Score** as the primary metric for selecting the best model. A higher R^2 indicates better predictive performance. MAE should also be considered as a supporting metric, where a lower value indicates lower prediction error.

Step 4: Compare the Runs

Use the MLflow interface to compare the recorded runs.

Complete the following table:

Run	Configuration	MAE	R^2	Selected?
1				
2				
3				
4				

Observe

Identify the run having the most suitable performance.

For this experiment: **Higher R^2 and lower MAE indicate better model performance.**

Step 5: Register the Model

Modify the existing MLFlow tracking program or generate a new one so that the trained model is registered in MLflow Model Registry with the name HousePriceModel. Make sure to retain the existing experiment tracking.

Step 6: Run the Selected Configuration

Use the configuration corresponding to the best-performing model identified earlier.

Run the program again to log the model and register it in the MLFlow registry.

Step 7: Open the Model Registry
In the MLflow interface, open the registered model. Observe the model name

Step 8: Create a New Model Version
Modify one experimental configuration (any hyperparameter) and run the program again.

Step 9: Compare Model Versions
Use the MLflow Model Registry to examine the available versions.
Compare their associated runs and performance metrics.

6. Expected Result
Multiple MLflow runs are compared using evaluation metrics, the best-performing model is identified, and the selected model is registered and maintained using the MLflow Model Registry.

8. Viva-Voce Questions
1. Why is model selection required?
2. Which metric was used to select the best model?
3. Why is R^2 useful for regression?
4. What is the MLflow Model Registry?
5. What is the difference between an MLflow run and a registered model?
6. What is a model version?
7. Why is model versioning important?
8. Can multiple versions of the same model be registered?
9. How does the Model Registry help in MLOps?
10. What is the difference between experiment tracking and model management?

4. Use MLflow to retrieve a saved experiment details and reproduce the model execution.

1. Aim

To retrieve the details of a previously recorded machine learning experiment from MLflow and reproduce the corresponding model execution.

2. Objectives

After completing this experiment, students will be able to:

- Retrieve a previously recorded MLflow run.
- Reproduce a machine learning experiment using the recorded configuration.
- Understand the role of MLflow in reproducibility.

4. Prerequisite

Use the **house price prediction project and MLflow experiment created in Programs 1–3**.
At least one completed MLflow run containing parameters, metrics and a saved model should be available.

5. Procedure

Step 1: Open the MLflow Interface

Start the MLflow tracking interface.
Open the **House Price Prediction** experiment.

Step 2: Select a Previous Run

Select any completed run from the experiment.
Preferably, select the best-performing run identified in Program 3.

Observe

Record the following information from the selected run:

- Run ID
- Parameters
- Metrics
- Model artifact

Step 4: Create a Reproduction Program

Create a simple Python program that reproduces the selected house price prediction experiment using the California Housing dataset. Use the same train-test split, preprocessing technique and model configuration recorded in the MLflow run. Train the model again and calculate MAE and R^2.

Step 5: Run the Reproduction Program

Execute the generated Python program.

Observe

Record the newly obtained:

- MAE
- R^2

Compare these values with those recorded in MLflow.

The reproduced model should produce performance close to the original run when the same dataset, preprocessing, model configuration and random state are used. (Small differences may occur depending on the implementation and environment.)

Step 8: Verify the Model Artifact
Return to the selected MLflow run and open its artifacts.
Locate the saved model. Observe that the MLflow stores not only the numerical results but also the trained model associated with the experiment.

7. Expected Result
The details of a previously recorded MLflow experiment are successfully retrieved and used to reproduce the machine learning model execution. The original and reproduced model performance are compared.

9. Viva-Voce Questions
1. What is reproducibility in machine learning?
2. Why is reproducibility important in MLOps?
3. What information is required to reproduce an experiment?
4. What is a Run ID?
5. How can parameters be retrieved from an MLflow run?
6. What is a model artifact?
7. What is the difference between reproducing an experiment and creating a new experiment?
8. Why should the random state be recorded?
9. Can a trained model be retrieved from MLflow?
10. How does MLflow support reproducible machine learning?

MODULE-4: AUTOMATED ML PIPELINES

1. Develop a Flask-based REST API to expose a trained ML model for prediction requests through external applications like postman.

1. Aim

To develop a Flask-based REST API that exposes a trained machine learning model and accepts prediction requests from external applications such as Postman.

2. Objectives

After completing this experiment, students will be able to:

- Develop a REST API using Flask.
- Create an endpoint for machine learning predictions.
- Accept input data through a POST request.
- Test the API using Postman.

5. Procedure

Step 1: Create the Project

Create a folder named:

ml-flask-api

Step 2: Generate the Flask API code for exposing a trained machine learning model.

The generated program should contain:

Flask Application

↓

/ endpoint

↓

/predict endpoint

↓

Receive JSON

↓

Model Prediction

↓

Return JSON Response

Step 3: Run the Flask Application

Step 4: Test the Root Endpoint

Open the Flask URL in a browser.

Step 6: Test the Prediction Endpoint Using Postman

Open Postman.

Create a:

POST

request to:

/predict

Select:

Body → raw → JSON

Provide suitable housing feature values.

Observe

The API should return a JSON response containing the predicted house price.

For example:

```
{
   "prediction": 2.45
}
```

The exact prediction will depend on the model and input values.

6. Expected Result

A Flask-based REST API is successfully developed and tested using Postman. The API accepts housing-related input data and returns the prediction generated by the trained machine learning model.

8. Viva-Voce Questions

1. What is a REST API?
2. Why is Flask used in this experiment?
3. What is an endpoint?
4. Why is POST used for prediction requests?
5. What is JSON?
6. How does Postman help in testing APIs?
7. What is the difference between GET and POST?
8. How is the trained ML model integrated with Flask?

2. Develop a Streamlit-based user-friendly web interface for a ML model that accepts user inputs, displays prediction results, and package the complete application using Docker for portable deployment.

1. Aim

To develop a Streamlit-based user interface for a machine learning model and package the complete application using Docker for portable deployment.

2. Objectives

After completing this experiment, students will be able to:

- Develop a simple Streamlit interface for an ML model.
- Accept user inputs through a web interface.
- Display prediction results.
- Create a Docker image.
- Run an ML application inside a Docker container.

4. Prerequisite

Use the **house price prediction model and project developed in Program 1**.

5. Procedure

Step 1: Create the Streamlit Application

Create:

streamlit_app.py

Step 2: Generate the Streamlit Interface for the house price prediction model.

The application should provide:

Input Features

↓

Predict Button

↓

ML Model

↓

Predicted House Price

Step 3: Run the Application Locally

Start the Streamlit application.

The application should open in a browser.

Enter values and click **Predict**.

Verify that the prediction is displayed.

Step 4: Create the Requirements File

Create a requirements.txt file containing only the Python packages required to run this Streamlit machine learning application.

Step 5: Create the Dockerfile for this project.

Step 6: Build the Docker Image

Step 7: Run the Docker Container
Observe
Open the Streamlit application in a browser.
Verify that the application is running from inside the container.

Step 8: Test the Containerized Application
Enter different property values and generate predictions. The application should successfully generate predictions from within the Docker container.

NOTE: Push the completed project to the GitHub repository. Build the Docker image and push it to the Docker Hub repository

6. Expected Result
A Streamlit-based machine learning application is successfully developed and packaged as a Docker image. The application runs inside a Docker container and generates predictions through a web interface.

8. Viva-Voce Questions
1. What is Streamlit?
2. Why is Streamlit useful for ML applications?
3. What is Docker?
4. What is a Docker image?
5. What is a Docker container?
6. What is a Dockerfile?
7. Why is requirements.txt required?
8. Why is port mapping required?
9. What is the advantage of containerizing an ML application?

3. Implement direct deployment strategy by replacing an existing Docker container with an updated version of a machine learning application.

1. Aim

To implement a direct deployment strategy by replacing an existing Docker container with an updated version of a machine learning application.

2. Objectives

After completing this experiment, students will be able to:

- Modify an existing ML application.
- Build an updated Docker image.
- Stop and remove an existing container.
- Deploy the updated image.
- Verify the updated application.

4. Prerequisite

Use the **Dockerized Streamlit application developed in Program 2**.

5. Procedure

Step 1: Modify the Application

Modify the current Streamlit house price prediction application to add a visible application version number and a short description below the title. Do not change the prediction logic.

Step 3: Test the Updated Application

Run the modified application locally.

Step 4: Build a New Docker Image

Build a new Docker image using a different tag.
For example:
house-price:v2

Step 5: Stop the Existing Container

Step 6: Remove the Existing Container

Step 7: Run the Updated Container

Step 8: Verify the Updated Application

Open the application in a browser.
Verify:

- Updated version information is displayed.
- Prediction functionality works.
- The application is running from the new container.

6. Expected Result

The existing machine learning application is successfully updated by creating a new Docker image and replacing the existing container with the updated version.

8. Viva-Voce Questions

1. What is direct deployment?
2. Why is a new Docker image created after modifying the application?
3. What is an image tag?
4. What is the difference between an image and a container?
5. Why is the existing container stopped?
6. Why is the old container removed?
7. What are the limitations of direct deployment?
8. What could happen to users while the container is being replaced?

4. Configure a GitHub Actions workflow to automate the process of building and validation of the Docker image of a ML project whenever changes are pushed to the repository.

1. Aim

To configure GitHub Actions to automatically build and push the Docker image of a machine learning application to Docker Hub whenever changes are pushed to the GitHub repository.

2. Objectives

1. Create a Docker Hub repository for the machine learning application.
2. Generate a Docker Hub Access Token for authentication.
3. Configure GitHub Repository Secrets for secure authentication.
4. Create a GitHub Actions workflow to build and push the Docker image.
5. Automate the publishing of updated Docker images to Docker Hub.

4. Software / Tools Required

- Visual Studio Code
- Git
- GitHub
- Docker Desktop
- Docker Hub
- GitHub Actions
- Existing Dockerized Streamlit ML application from Program 2

5. Procedure

Part A – Create a Docker Hub Repository

- ✓ Sign in to Docker Hub.
- ✓ From the Docker Hub dashboard, select Repositories and click create repository.
- ✓ Enter the repository name as: house-price-ml
- ✓ Select the required visibility and click Create.
- ✓ The Docker image will later be available in the format: DOCKERHUB_USERNAME/house-price-ml:latest

Part B – Create Docker Hub Access Token

- ✓ Sign in to Docker Hub.
- ✓ Click the profile icon in the top-right corner. Select Account Settings.
- ✓ From the left-side menu, select Personal access tokens. Click Generate new token.
- ✓ Enter the token description as: github-actions-token
- ✓ Select the permission as Read, Write, Delete to push images to the Docker Hub repository. Click Generate.
- ✓ Copy the generated token and store it temporarily in a secure location.

Part C – Configure GitHub Repository Secrets

- ✓ Open the GitHub repository containing the Dockerized ML application.
- ✓ Click Settings. From the left-side menu, select Secrets and variables.
- ✓ Select Actions. Under Repository secrets, click New repository secret.
- ✓ Enter the following details:

  Name: DOCKERHUB_USERNAME

  Secret: Enter your Docker Hub username.

- ✓ Click Add secret.
- ✓ Click New repository secret again.
- ✓ Enter the following details:
  Name: DOCKERHUB_TOKEN
  Secret: Paste the Docker Hub Access Token generated in Part B.
- ✓ Click Add secret.

The repository should now contain the following secrets:

Secret Name	Value
DOCKERHUB_USERNAME	**Docker Hub username**
DOCKERHUB_TOKEN	**Docker Hub Access Token**

Part D – Create GitHub Actions Workflow

Create a GitHub Actions workflow for a Dockerized Streamlit machine learning project. The workflow should run whenever code is pushed to the GitHub repository.
Create the workflow file. The workflow file should be created inside .github/workflows/
For example:
.github/workflows/docker-publish.yml

Part F – Execute the GitHub Actions Workflow

- ✓ Open the GitHub repository.
- ✓ Select the Actions tab.
- ✓ Select the Docker image workflow.
- ✓ Open the latest workflow run.
- ✓ Check the workflow steps for successful completion.
- ✓ Open Docker Hub.
- ✓ Navigate to Repositories → house-price-ml.
- ✓ Verify that the Docker image with the latest tag has been pushed.

Part I – Demonstrate Automatic Image Update

Open the Streamlit application in Visual Studio Code.
Modify the current Streamlit house price prediction application to change the application title to “House Price Prediction – Version 2”.
Step 1: Test the application locally.
Step 2: Commit and push the changes to GitHub.
Step 3: Open the Actions tab in GitHub.
Step 4: Open the newly triggered workflow.
Step 5: Verify that the Docker image is built and pushed to Docker Hub.
Step 6: Open Docker Hub → Repositories → house-price-ml.
Step 7: Verify that the latest image has been updated.
Step 8: Pull and run the updated Docker image.
Step 9: Open the application and verify the updated title.

6. Expected Result

The GitHub Actions workflow successfully builds the Docker image of the Streamlit machine learning application and pushes the image to Docker Hub whenever changes are pushed to the GitHub repository.

Viva-Voce Questions

1. What is Docker Hub?
2. What is a Docker Hub Access Token?
3. Why is an Access Token used instead of a Docker Hub password?
4. What are GitHub Repository Secrets?
5. Where are Repository Secrets created in GitHub?
6. Why should Docker Hub credentials not be written directly in the workflow file?
7. What event triggers the workflow?
8. What is the purpose of the Docker build operation?
9. What is the purpose of the Docker push operation?
10. What is the purpose of the latest tag?
11. What is the role of GitHub Actions in this experiment?
12. What happens when a developer pushes a new change to GitHub?
13. How can the Docker image stored in Docker Hub be used on another computer?
14. How does this experiment demonstrate CI/CD?

MODULE-5: MODEL DEPLOYMENT & MONITORING

1. Create and configure an AWS EC2 virtual machine and host a containerized application.

1. Aim

To create and configure an AWS EC2 virtual machine and host a simple containerized application using Docker.

2. Objective

1. Create an EC2 virtual machine instance on AWS.
2. Configure security groups and network access.
3. Connect to an EC2 instance using SSH.
4. Install and configure Docker on an EC2 instance.
5. Build a Docker image for a simple application.
6. Run a Docker container on the EC2 instance.
7. Verify that the containerized application is running.

3. Software and Tools Required

- AWS Account
- Amazon EC2
- Ubuntu Server
- Docker
- SSH Client (PowerShell / PuTTY)
- GitHub
- Browser

4. Procedure

Step 1: Create an AWS EC2 instance

1. Sign in to the AWS Management Console.
2. Open **EC2**.
3. Select **Launch Instance**.
4. Enter an appropriate instance name, for example: mlops-module5-lab
5. Select an Ubuntu Server AMI.
6. Select a small instance type suitable for the lab.
7. Create/select a key pair.
8. Configure the network/security group.
   SSH TCP 22
   HTTP TCP 80
   Custom TCP 5000
9. Launch the instance.

Step 2: Obtain the Public IP Address

From the EC2 console, note the instance's:
Public IPv4 address

Step 3: Connect to EC2 using SSH

Open PowerShell

- Run: ssh -i path\to\key.pem ec2-user@<Public IP address>

- Ensure your .pem file has correct permissions (read-only)
- Accept the fingerprint prompt to connect

Step 4: Update the packages

1. sudo apt update
2. sudo apt upgrade -y

Step 5: Install Docker

sudo apt install docker.io -y

Start Docker:
sudo systemctl start docker

Enable Docker:
sudo systemctl enable docker

Verify:
docker --version

Step 6: Allow the Ubuntu user to execute Docker

sudo usermod -aG docker $USER

Log out and reconnect to the EC2 instance.

Verify:
docker ps

Step 7: Create a Simple Application

1. Create a directory:
   mkdir mlops-app
   cd mlops-app

2. Create app.py:

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "MLOps Application Running on AWS EC2!"

app.run(host="0.0.0.0", port=5000)
```

3. Create requirements.txt:
   flask

Step 8: Create Dockerfile

1. Create a file named:

Dockerfile

2. Contents:

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

Step 9: Build the Docker Image

docker build -t mlops-app .

Verify:

docker images

Expected result:

mlops-app

Step 10: Run the Container

docker run -d -p 5000:5000 --name mlops-container mlops-app

Check:

docker ps

Step 11: Test the Application

From the EC2 server:

curl http://localhost:5000

Expected output:

MLOps Application Running on AWS EC2!

5. Viva-Voce Questions

1. What is AWS EC2?
2. What is the difference between an EC2 instance and a Docker container?
3. What is an AMI?
4. What is a security group?
5. Why is port 22 required?
6. Why is port 5000 opened in this experiment?
7. What is Docker?
8. What is a Docker image?
9. What is a Docker container?

10. What is the purpose of a Dockerfile?
11. What does docker build do?
12. What does docker run do?
13. What is the purpose of -p 5000:5000?
14. Why do we use 0.0.0.0 instead of localhost in the Flask application?
15. What happens if port 5000 is not allowed in the security group?

2. Deploy a containerized machine learning application on an AWS EC2 instance using Docker.

1. Aim

To deploy a containerized machine learning inference application on an AWS EC2 instance using Docker.

2. Objective

- Develop a simple ML inference API.
- Package the ML application using Docker.
- Build a Docker image.
- Run the image as a container on EC2.
- Send input data to the deployed model.
- Obtain predictions from the model.

3. Software and Tools Required

- AWS EC2
- Ubuntu
- Python
- Flask
- scikit-learn
- Docker
- Git/GitHub
- Postman/CURL

4. Procedure

Step 1: Prepare the ML Application

Create:

```
ml-model-app/
 |
 ├── app.py
 ├── model.py
 ├── requirements.txt
 └── Dockerfile
```

Step 2: Create the Machine Learning Model

For demonstration, use the Iris dataset.

model.py:

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression

iris = load_iris()

model = LogisticRegression(max_iter=200)
model.fit(iris.data, iris.target)

def predict(features):
```

```
    prediction = model.predict([features])
    return int(prediction[0])
```

Step 3: Create the Inference API

app.py:

```
from flask import Flask, request, jsonify
from model import predict

app = Flask(__name__)

@app.route("/predict", methods=["POST"])
def prediction():

    data = request.get_json()

    features = data["features"]

    result = predict(features)

    return jsonify({
        "prediction": result
    })

@app.route("/")
def home():
    return "ML Prediction API is running!"

app.run(host="0.0.0.0", port=5000)
```

Step 4: Create requirements.txt

```
flask
scikit-learn
numpy
```

Step 5: Create Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

Step 6: Build Docker Image

```
docker build -t iris-ml-api .
```

Verify:

```
docker images
```

Step 7: Run the ML Container

```
docker run -d \
-p 5000:5000 \
--name iris-ml-container \
iris-ml-api
```

Check:

```
docker ps
```

Step 8: Test the Prediction API

1. Get Your EC2 Public IP
   - Log in to AWS Management Console
   - Go to EC2 → Instances
   - Copy the Public IPv4 address or Public DNS
2. Verify Security Group Rules
   - Ensure your EC2 allows inbound traffic on the API port.
   - In EC2 console, select your instance
   - Go to Security → Security Groups
   - Add inbound rule for TCP port (e.g., 5000/8000/443)
   - Source: 0.0.0.0/0 for testing
3. Test with Postman
   - Postman provides a GUI for API testing.
   - Open Postman
   - Create a new POST request
   - Enter URL: http://<EC2-IP>:<PORT>/predict
   - In Body → raw → JSON, enter:
     {"features":[5.1,3.5,1.4,0.2]}
4. Expected response:
   - {
   - "prediction": 0
   - }

Step 9: Examine Container Logs

```
docker logs iris-ml-container
```

5. Viva-Voce Questions

1. What is ML model deployment?
2. What is inference?
3. What is an inference API?
4. Why do we use Flask in this experiment?

5. Why is Docker useful for ML deployment?
6. What is the difference between a model and an inference service?
7. What does the /predict endpoint do?
8. Why do we use POST instead of GET for prediction?
9. What is the purpose of requirements.txt?
10. What happens when the Docker container is stopped?
11. Where is the ML model executed?
12. What is the advantage of containerizing an ML application?
13. What is the difference between training and inference?
14. What is model serving?
15. What is the role of EC2 in this architecture?

3. Verify Deployed Application Accessibility Through the Server's Public IP Address

1. Aim
To verify the accessibility of a containerized machine learning application deployed on an AWS EC2 instance using its public IP address.

2. Objective

- Identify the public IP address of an EC2 instance.
- Understand inbound network traffic.
- Configure AWS security groups.
- Access a containerized ML application remotely.
- Test an inference API from an external system.
- Troubleshoot common connectivity problems.

3. Software and Tools Required

- AWS EC2
- Docker
- Browser
- Postman
- Flask ML application from Exercise 2

4. Procedure
Step 1: Verify the Container
On EC2:
`docker ps`

Step 2: Verify Local Access
Execute:
curl http://localhost:5000

This confirms that the application works inside the EC2 instance.

Step 3: Obtain Public IP
Get Your EC2 Public IP

1. Log in to AWS Management Console
2. Go to EC2 → Instances
3. Copy the Public IPv4 address or Public DNS

Step 4: Configure Security Group
Go to:

EC2
→ Security
→ Security Groups
→ Inbound Rules

Add:

Type: Custom TCP
Port: 5000
Source: appropriate testing source

Step 5: Test Through Browser

Open:

http://<PUBLIC-IP>:5000

Expected:

ML Prediction API is running!

Step 6: Test Using Postman

Create:

POST
http://<PUBLIC-IP>:5000/predict

Select:

Body → raw → JSON

Enter:

```
{
  "features": [5.1, 3.5, 1.4, 0.2]
}
```

Click **Send**.

5. Viva-Voce Questions

1) What is a public IP address?
2) What is the difference between a private IP and public IP?
3) What is an AWS security group?
4) What is an inbound rule?
5) Why can't the application be accessed externally even though localhost:5000 works?
6) What is port mapping in Docker?
7) What does 0.0.0.0:5000->5000 mean?
8) Why is 0.0.0.0 used by the Flask application?
9) Why should ports not unnecessarily be exposed to 0.0.0.0/0?
10) How would you make the application more secure?
11) What is an Elastic IP?
12) What happens to an EC2 public IP after stopping and restarting an instance?
13) What is the role of Postman?
14) How would you troubleshoot a connection timeout?
15) How can HTTPS be enabled for a production ML API?

4. Deploy a Containerized Machine Learning Application on a Kubernetes Cluster Hosted on AWS EC2

1. Aim
To deploy and expose a containerized machine learning application using Kubernetes on an AWS EC2-based Kubernetes cluster.

2. Objective
1) Understand basic Kubernetes architecture.
2) Create a Kubernetes cluster on EC2.
3) Deploy a Dockerized ML application using a Kubernetes Deployment.
4) Create a Kubernetes Service.
5) Expose the ML application.
6) Verify the deployed application.

3. Software and Tools Required
- AWS EC2
- Ubuntu
- Docker
- Kubernetes
- Minikube or lightweight Kubernetes
- kubectl
- Flask
- Docker Hub

4. Procedure
Step 1: Prepare EC2
Create an Ubuntu EC2 instance.

Install Docker:
- sudo apt update

- sudo apt-get install -y docker.io
- sudo systemctl enable docker
- sudo systemctl start docker

Verify:
docker --version

Step 2: Install kubectl
Install the Kubernetes command-line utility appropriate for the selected Ubuntu environment.

- curl -LO "https://dl.k8s.io/release/$(curl -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
- chmod +x kubectl
- sudo mv kubectl /usr/local/bin/

- kubectl version --client

Verify:
kubectl version --client

Step 3: Install Minikube
Install Minikube following the current installation method appropriate for the EC2 Ubuntu environment.

- curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
- sudo install minikube-linux-amd64 /usr/local/bin/minikube
- Verify: minikube version

Step 4: Start Kubernetes Cluster
For a Docker-based Minikube cluster:

```
minikube start --driver=docker
```

Verify:

```
kubectl get nodes
```

Step 5: Obtain the ML Docker Image

The ML application created in Exercise 2 should be available as a container image.

For example:

```
yourdockerhubusername/iris-ml-api:1.0
```

Students can push their image to Docker Hub:

```
docker login
```

Then:

```
docker tag iris-ml-api yourdockerhubusername/iris-ml-api:1.0
docker push yourdockerhubusername/iris-ml-api:1.0
```

Step 6: Create Kubernetes Deployment

Create:

```
deployment.yaml
```

Example:

```
apiVersion: apps/v1
kind: Deployment
```

```
metadata:
  name: iris-ml-deployment

spec:
  replicas: 2

  selector:
    matchLabels:
      app: iris-ml

  template:
    metadata:
      labels:
        app: iris-ml

    spec:
      containers:
        - name: iris-ml-container
          image: yourdockerhubusername/iris-ml-api:1.0
          ports:
            - containerPort: 5000
```

Step 7: Create the Deployment

```
kubectl apply -f deployment.yaml
```

Verify:

```
kubectl get deployments
```

Then:

```
kubectl get pods
```

Expected:

```
iris-ml-deployment-xxxxx
iris-ml-deployment-yyyyy
```

Step 8: Create Kubernetes Service

Create:

```
service.yaml
apiVersion: v1
kind: Service

metadata:
  name: iris-ml-service

spec:
  selector:
    app: iris-ml
```

```
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5000

  type: NodePort
```

Apply:

```
kubectl apply -f service.yaml
```

Verify:

```
kubectl get services
```

Step 9: Access the Application

Use:

```
minikube service iris-ml-service --url
```

This returns an accessible URL.

Students can test:

```
curl <SERVICE-URL>
```

Expected:

```
ML Prediction API is running!
```

Step 10: Test Prediction

```
curl -X POST <SERVICE-URL>/predict \
-H "Content-Type: application/json" \
-d '{"features":[5.1,3.5,1.4,0.2]}'
```

Expected:

```
{
    "prediction": 0
}
```

5. Viva-Voce Questions

1) What is Kubernetes?
2) Why is Kubernetes required when Docker can already run containers?
3) What is a Kubernetes cluster?
4) What is a Node?
5) What is a Pod?
6) Can a Pod contain multiple containers?

7) What is a Deployment?
8) What is a Kubernetes Service?
9) What is kubectl?
10) What is Minikube?
11) What is the purpose of deployment.yaml?
12) What does replicas: 2 mean?
13) What happens when a Pod fails?
14) How does Kubernetes maintain the desired number of replicas?
15) What is the difference between a Docker container and a Kubernetes Pod?
16) Why is Kubernetes useful for ML model deployment?
17) How can Kubernetes help scale an ML inference service?
18) How would you deploy a new version of a model?
19) What is rolling deployment?